

We now delve into the main function $parse(f, \{c_i\})$: we first decode the root operation op of the hierarchy by $OpDecoder(f)$. If op requires a concept parameter (an object-level concept, a relational concept, or an attribute), $ConceptDecoder$ will be invoked to choose a concept from all concepts $\{c_i\}$. Assuming op takes two non-concept inputs (e.g., the operation `Intersection` takes two object sets as its input), there will be two branches for this root node. Thus, two output encoders $OEncoder_0$ and $OEncoder_1$ will be applied to transform the current state f into two sub-states f_1 and f_2 . $parse$ will be recursively invoked based on f_1 and f_2 to generate the two branches respectively. In the DSL, the number of non-concept inputs for any operation is at most 2.

In our implementation, the input encoder $IEncoder$ first maps each word in the question into an embedding space. The word embeddings are composed of two parts: a randomly initialized word embedding of dimension 256 and a positional embedding of dimension 128 (Gehring et al., 2017). For a *concept word*, its word embedding only depends on which type it belongs to (i.e. object-level, relational or attribute). Thus, after being trained on a fixed dataset, the semantic parser can parse questions with novel (unseen) *concept words*. The sequence of word embeddings is then encoded by a two-layer GRU with a hidden dimension of $256 * 2$ (bidirectional). The function $parse$ starts from the last hidden state of the GRU, and works recursively to generate the hierarchical program layout. Both $OpDecoder$ and $ConceptDecoder$ are feed-forward networks. $ConceptDecoder$ performs attentions over the representations of all concepts $\{c_i\}$ to select the concepts. Output encoders $OEncoder_0$ and $OEncoder_1$ are implemented as GRU cells.

Another pre-processing of the sentence is to group consecutive object-level *concept words* into a group and treat them together as a single concept, inspired by the notion of “noun phrases” in natural languages. The computational intuition behind this grouping is that, the latent programs of CLEVR questions usually contain multiple consecutive `Filter` tokens. During the program parsing and execution, we aim to fuse all such `Filters` into a single `Filter` operation that takes multiple concepts as its parameter.

A Running Example As a running example, consider again the question “What is the color of the cube right of the red matte object?”. We first process the sentence (by rules) as: “What is the <Attribute 1 (color)> of the <(ObjConcept 1 (cube))> <RelConcept 1 (right)> of the <ObjConcept 2 (red matte object)>?”. The expected parsing result of this sentence is:

```
Query(<Attribute 1>,
      Filter(<ObjConcept 1>,
            Relate(<RelConcept 1>,
                  Filter(<ObjConcept 2>, Scene)
                )
          )
    ).
```

The semantic parser encode the word embeddings with $IEncoder$. The last hidden state of the GRU will be used as f_0 . The word embeddings of the *concept words* form the set $\{c_i\} = \{\text{Attribute 1, ObjConcept 1, RelConcept 1, ObjConcept 2}\}$. The function $parse$ is then invoked recursively to generate the hierarchical program layout. Table 8 illustrates the decoding process step-by-step.

C PROGRAM EXECUTION

In this section, we present the implementation of all operations listed in Table 6. We start from the implementation of Object-typed and ObjectSet-typed variables. Next, we discuss how to classify objects by object-level concepts or relational concept, followed by the implementation details of all operations.

Object-typed and ObjectSet-typed variables. We consider a scene with n objects. An Object-typed variable can be represented as a vector `Object` of length n , where $Object_i \in [0, 1]$ and $\sum_i Object_i = 1$. $Object_i$ can be interpreted as the probability that the i -th object of the scene is being referred to. Similarly, an ObjectSet-typed variable can be represented as a vector `ObjectSet` of length n , where $ObjectSet_i \in [0, 1]$. $ObjectSet_i$ can be interpreted as the probability that the i -th object is in the set. To cast an ObjectSet-typed variable `ObjectSet` as an Object-typed variable

Step	Inputs	Outputs	Recursive Invocation
1	f_0	OpDecoder(f_0) $\rightarrow$ Query; ConceptDecoder(f_0) $\rightarrow$ < Attribute 1 >; OEncoder ₀ (f_0 , Query) $\rightarrow f_1$	$parse(f_1)$
2	f_1	OpDecoder(f_1) $\rightarrow$ Filter; ConceptDecoder(f_1) $\rightarrow$ < ObjConcept 1 >; OEncoder ₀ (f_1 , Filter) $\rightarrow f_2$	$parse(f_2)$
3	f_2	OpDecoder(f_2) $\rightarrow$ Relate; ConceptDecoder(f_2) $\rightarrow$ < RelConcept 1 >; OEncoder ₀ (f_2 , Relate) $\rightarrow f_3$	$parse(f_3)$
4	f_3	OpDecoder(f_3) $\rightarrow$ Filter; ConceptDecoder(f_3) $\rightarrow$ < ObjConcept 2 >; OEncoder ₀ (f_3 , Filter) $\rightarrow f_4$	$parse(f_4)$
5	f_4	OpDecoder(f_3) $\rightarrow$ Scene;	(End of branch.)

Table 8: A step-by-step running example of the recursive parsing procedure. The parameter $\{c_i\}$ is omitted for better visualization.

Object (i.e., the Unique operation), we compute: $\text{Object} = \text{softmax}(\sigma^{-1}(\text{ObjectSet}))$, where $\sigma^{-1}(x) = \log(x/(1-x))$ is the logit function.

Concept quantization. Denote o_i as the visual representation of the i -th object, OC the set of all object-level concepts, and A the set of all object-level attributes. Each object-level concept oc (e.g., Red) is associated with a vector embedding v^{oc} and a L1-normalized vector b^{oc} of length $|A|$. b^{oc} represents which attribute does this object-level concept belong to (e.g., the concept Red belongs to the attribute Color). All attributes $a \in A$ are implemented as neural operators, denoted as u^a (e.g., u^{Color}). To classify the objects as being Red or not, we compute:

$$\Pr[\text{object } i \text{ is Red}] = \sigma \left(\sum_{a \in A} \left(b_a^{\text{Red}} \cdot \frac{\langle u^a(o_i), v_{\text{Red}} \rangle - \gamma}{\tau} \right) \right),$$

where σ denotes the Sigmoid function, $\langle \cdot, \cdot \rangle$ the cosine distance between two vectors. γ and τ are scalar constants for scaling and shifting the values of similarities. By applying this classifier on all objects we will obtain a vector of length n , denoted as $\text{ObjClassify}(\text{Red})$. Similarly, such classification can be done for relational concepts such as Left. This will result in an $n \times n$ matrix $\text{RelClassify}(\text{Left})$, where $\text{RelClassify}(\text{Left})_{j,i}$ is the probability that the object i is left of the object j .

To classify whether two objects have the same attribute (e.g., have the same Color), we compute:

$$\Pr[\text{object } i \text{ has the same Color as object } j] = \sigma \left(\frac{\langle u^{\text{Color}}(o_i), u^{\text{Color}}(o_j) \rangle - \gamma}{\tau} \right),$$

We can obtain a matrix $\text{AEClassify}(\text{Color})$ by applying this classifier on all pairs of objects, where $\text{AEClassifier}(\text{Color})_{j,i}$ is the probability that the object i and j have the same Color.

Quasi-symbolic program execution. Finally, Table 9 summarizes the implementation of all operators. In practice, all probabilities are stored in the log space for better numeric stability.

D OPTIMIZATION OF THE SEMANTIC PARSER

To tackle the optimization in a non-smooth program space, we apply an off-policy program search process (Sutton et al., 2000) to facilitate the learning of the semantic parser. Denote $\mathbb{P}(s)$ as the set of all valid programs in the CLEVR DSL for the input question s . We want to compute the gradient w.r.t. Θ_s , the parameters of the semantic parser:

$$\nabla_{\Theta_s} = \nabla_{\Theta_s} \mathbb{E}_P[r \cdot \log \Pr[P]],$$

Signature	Implementation
$\text{Scene}() \rightarrow \text{out}: \text{ObjectSet}$	$\text{out}_i := 1$
$\text{Filter}(\text{in}: \text{ObjectSet}, \text{oc}: \text{ObjConcept}) \rightarrow \text{out}: \text{ObjectSet}$	$\text{out}_i := \min(\text{in}_i, \text{ObjClassify}(\text{oc})_i)$
$\text{Relate}(\text{in}: \text{Object}, \text{rc}: \text{RelConcept}) \rightarrow \text{out}: \text{ObjectSet}$	$\text{out}_i := \sum_j (\text{in}_j \cdot \text{RelClassify}(\text{rc})_{j,i})$
$\text{AERelate}(\text{in}: \text{Object}, \text{a}: \text{Attribute}) \rightarrow \text{out}: \text{ObjectSet}$	$\text{out}_i := \sum_j (\text{in}_j \cdot \text{AECClassify}(\text{a})_{j,i})$
$\text{Intersection}(\text{in}^{(1)}: \text{ObjectSet}, \text{in}^{(2)}: \text{ObjectSet}) \rightarrow \text{out}: \text{ObjectSet}$	$\text{out}_i := \min(\text{in}_i^{(1)}, \text{in}_i^{(2)})$
$\text{Union}(\text{in}^{(1)}: \text{ObjectSet}, \text{in}^{(2)}: \text{ObjectSet}) \rightarrow \text{out}: \text{ObjectSet}$	$\text{out}_i := \max(\text{in}_i^{(1)}, \text{in}_i^{(2)})$
$\text{Query}(\text{in}: \text{Object}, \text{a}: \text{Attribute}) \rightarrow \text{out}: \text{ObjConcept}$	$\text{Pr}[\text{out} = \text{oc}] := \sum_i \text{in}_i \cdot \frac{\text{ObjClassify}(\text{oc})_i \cdot b_a^{\text{oc}}}{\sum_{\text{oc}'} \text{ObjClassify}(\text{oc}')_i \cdot b_a^{\text{oc}'}}$
$\text{AEQuery}(\text{in}^{(1)}: \text{Object}, \text{in}^{(2)}: \text{Object}, \text{a}: \text{Attribute}) \rightarrow b: \text{Bool}$	$b := \sum_i \sum_j (\text{in}_i^{(1)} \cdot \text{in}_j^{(2)} \cdot \text{AECClassify}(\text{a})_{j,i})$
$\text{Exist}(\text{in}: \text{ObjectSet}) \rightarrow b: \text{Bool}$	$b := \max_i \text{in}_i$
$\text{Count}(\text{in}: \text{ObjectSet}) \rightarrow i: \text{Integer}$	$i := \sum_i \text{in}_i$
$\text{CLessThan}(\text{in}^{(1)}: \text{ObjectSet}, \text{in}^{(2)}: \text{ObjectSet}) \rightarrow b: \text{Bool}$	$b := \sigma((\sum_i \text{in}_i^{(2)} - \sum_i \text{in}_i^{(1)} - 1 + \gamma_c) / \tau_c)$
$\text{CGreaterThan}(\text{in}^{(1)}: \text{ObjectSet}, \text{in}^{(2)}: \text{ObjectSet}) \rightarrow b: \text{Bool}$	$b := \sigma((\sum_i \text{in}_i^{(1)} - \sum_i \text{in}_i^{(2)} - 1 + \gamma_c) / \tau_c)$
$\text{CEqual}(\text{in}^{(1)}: \text{ObjectSet}, \text{in}^{(2)}: \text{ObjectSet}) \rightarrow b: \text{Bool}$	$b := \sigma((- \sum_i \text{in}_i^{(1)} - \sum_i \text{in}_i^{(2)} + \gamma_c) / (\gamma_c \cdot \tau_c))$

Table 9: All operations in the domain-specific language for CLEVR VQA. $\gamma_c = 0.5$ and $\tau_c = 0.25$ are constants for scaling and shift the probability. During inference, one can quantify all operations as Yi et al. (2018).

where $P \sim \text{SemanticParse}(s; \Theta_s)$. In REINFORCE, we approximate this gradient via Monte Carlo sampling.

An alternative solution is to exactly compute the gradient. Note that in the definition of the reward r , only the set of programs $\mathbb{Q}(s)$ leading to the correct answer will contribute to the gradient term. With the perception module fixed, the set $\mathbb{Q}$ can be efficiently determined by an off-policy exhaustive search of all possible programs $\mathbb{P}(s)$. In the third stage of the curriculum learning, we search for the set $\mathbb{Q}$ offline based on the quantified results of concept classification and compute the exact gradient $\nabla \Theta_s$. An intuitive explanation of the off-policy search is that, we enumerate all possible programs, execute them on the visual representation, and find the ones leading to the correct answer. We use $\mathbb{Q}(s)$ as the “groundtruth” program annotation for the question, to supervise the learning, instead of running the Monte Carlo sampling-based REINFORCE.

Spurious program suppression. However, directly using $\mathbb{Q}(s)$ as the supervision by computing $\ell = \sum_{p \in \mathbb{Q}(s)} -\log \text{Pr}(p)$ can be problematic, due to the spuriousness or the ambiguity of the programs. This comes from two aspects:

1) intrinsic ambiguity: two programs are different but equivalent. For example

P1: $\text{AEQuery}(\text{Color}, \text{Filter}(\text{Cube}), \text{Filter}(\text{Sphere}))$ and
P2: $\text{Exist}(\text{Filter}(\text{Sphere}, \text{AERelate}(\text{Color}, \text{Filter}(\text{Cube}))))$

are equivalent.

2) extrinsic spuriousness: one of the program is incorrect, but also leads to the correct answer in a